
TD Terminaison

Question 1 : Soit l'algorithme suivant :

```
def f(a):  
    while a != 0:  
        a = a-1  
    return a
```

1. Cet algorithme termine-t-il pour toute entrée ? Si non, donnez un contre-exemple.
2. Si non, que faudrait-il rajouter à ce code pour être sûr qu'il termine ?

Question 2 : Prouvez la terminaison de la fonction suivante à l'aide d'un variant de boucle :

```
def somme(n):  
    res = 0  
    i = 0  
    while i < n:  
        res += i  
        i += 1  
    return res
```

Question 3 : On s'intéresse au calcul de la factorielle de manière récursive :

```
def factorielle(n):  
    if n == 0:  
        return 1  
    return n * factorielle(n - 1)
```

1. Donnez une pré-condition pour que la fonction termine sans erreur.
2. Prouvez la terminaison de cet algorithme à l'aide d'un variant.

Question 4 : On souhaite calculer x^n de manière récursive avec l'algorithme « d'exponentiation rapide » suivant :

```
def puissance(x, n):  
    if n == 0:  
        return 1  
    if n % 2 == 0:  
        return puissance(x * x, n // 2)  
    else:  
        return x * puissance(x, n - 1)
```

1. Donner les spécifications de la fonction.
2. Prouvez la terminaison de cet algorithme à l'aide d'un variant bien choisi. Vous vérifierez que chaque branche récursive fait bien décroître et validera la terminaison.

Question 5 : Soit l'algorithme de division euclidienne suivant :

```
def div_entiere(a, b):  
    q = 0  
    while a >= b:  
        a = a - b  
        q = q + 1  
    return q
```

1. Que se passe-t-il si b vaut 0 ?
2. Donnez des pré-conditions sur a et b pour que cet algorithme termine.
3. Justifiez la terminaison à l'aide d'un variant de boucle.

Question 6 : Indiquez si les algorithmes suivants terminent pour tout $n \in \mathbb{N}$. Si ce n'est pas le cas, donnez la plus petite valeur de n pour laquelle l'algorithme ne termine pas (ou qu'il plante), en justifiant.

Algorithme 1 :

```
def algo_1(n):  
    while n != 0:  
        n = n - 2  
    return n
```

Algorithme 2 :

```
def algo_2(n):  
    while n > 0:  
        n = n + 1  
    return n
```

Algorithme 3 :

```
def algo_3(n):  
    while n > 1:  
        if n % 2 == 0:  
            n = n // 2  
        else:  
            n = 3 * n + 1  
    return n
```

Question 7 : Considérez le programme suivant avec deux variables :

```
def mystere(m, n):  
    while m > 0 or n > 0:  
        if m > 0:  
            m = m - 1  
            n = n + 10  
        else:  
            n = n - 1
```

Prouvez que cet algorithme termine pour tout couple d'entiers naturels (m, n) . *Indication : il n'est pas possible de trouver une simple fonction variant à valeurs dans $\mathbb{N}$ qui décroît strictement à chaque itération. Cherchez plutôt à utiliser un tuple de variables et un ordre bien fondé dessus (par exemple l'ordre lexicographique).*